

Personal Finance Dashboard

Architecture Documentation (arc42)

Roman Lysser · Module WEBLAB.F2601 · 25. September 2026

Contents

1.	Introduction and Goals	3
1.1.	Requirements Overview	3
1.2.	Quality Goals	3
1.3.	Stakeholders	3
2.	Architecture Constraints	4
3.	System Scope and Context	5
3.1.	Business Context	5
3.2.	Technical Context	5
4.	Solution Strategy	6
4.1.	Technology and decomposition	6
4.2.	Quality goals	6
5.	Building Block View	7
5.1.	Level 1: Whitebox Overall System	7
5.2.	Level 2: Backend Architecture	8
5.3.	Level 2: Frontend Architecture	9
6.	Runtime View	11
6.1.	Sign-in and session	11
6.2.	Onboarding	12
6.3.	Monthly limits take-over	12
7.	Deployment View	13
7.1.	Runtime configuration	15
7.2.	Development seed	15
8.	Cross-Cutting Concepts	16
8.1.	Domain model	16
8.2.	Household scoping	16
8.3.	Domain errors at the HTTP boundary	16
8.4.	Testing	17
8.5.	Theming	18
8.6.	Security	18
8.7.	Money and account numbers	18
8.8.	Responsive shell and dialogs	18
8.9.	Quick transaction entry	19
8.10.	Route loading	19
9.	Architecture Decisions (ADRs)	20
9.1.	ADR-1: "Uncategorized" is the absence of a category, not a reserved category row	20

9.2.	ADR-2: A new month takes over the nearest earlier limits, automatically only for the current and next month	20
9.3.	ADR-3: Use libsql batch() for atomic multi-statement writes	24
9.4.	ADR-4: Mount better-auth's HTTP handler before Nest initializes	24
10.	Quality Requirements	26
10.1.	Quality Tree	26
10.2.	Quality Scenarios	26
10.3.	Lighthouse Results	27
11.	Risks and Technical Debt	29
12.	Glossary	30

1. Introduction and Goals

1.1. Requirements Overview

The dashboard lets a household record accounts and transactions, plan monthly category limits, and inspect balances and spending. `docs/proposal.typ` is the source for the story definitions. These IDs appear in code comments where a rule implements a specific story.

1.2. Quality Goals

Priority	Goal	Why
1	Atomic financial writes	A failed onboarding or category transfer must not leave a partial household or history.
2	Responsive navigation	First-page requests should finish before unrelated route chunks compete for bandwidth; phone and desktop layouts remain usable.
3	Low-effort transaction entry	Adding a transaction must take as little effort as possible, so the app never keeps the user from tracking transactions and budgets. Skipped entries make balances and limits drift from reality.

1.3. Stakeholders

Role	Expectation
Developer (Roman Lysser)	Well-thought-out architecture, tools that are clear to use, fast-running scripts, and a polished overall look and feel.
Lecturer (HSLU)	Minimum project requirements: • Functional: create, show, change, and delete a self-defined resource; store the data persistently in a database; present the data in at least two different forms. • Responsive: optimized for desktop as well as mobile and tablet. • Tested: meaningful automated unit, integration, and E2E tests. • Lighthouse: score of at least 90 (average of all audits) for mobile and desktop. • Reproducible: the production bundle starts reproducibly, via a public URL or one <code>docker compose up</code>. • Code: readable and extensible, with a sensible, well-thought-out structure of the whole application.
End user	A well-structured overview, ease of use, and reliability.

2. Architecture Constraints

Constraint	Background
TypeScript end to end	One language from database schema to view; the OpenAPI spec generates the frontend client (@hey-api/openapi-ts).
Angular 22 + NestJS 12	Stack fixed in the proposal; both give feature modules and dependency injection.
Bun	Package manager, script runner, and API runtime in both projects.
SQLite, no extra infrastructure	One database file via libsql; runs with docker compose up only.
No bank connection	Switzerland has no bank-agnostic API for reading balances and transactions, so transactions are entered by hand; balance = opening balance + transactions.
One currency per household	household.base_currency (CHF, EUR, USD, GBP); no exchange rates.
Single developer, course timeline	Module WEBLAB.F2601; scope bounded by the MoSCoW stories of the proposal and the lecturer's minimum requirements (ch. 1 Stakeholders).

3. System Scope and Context

3.1. Business Context

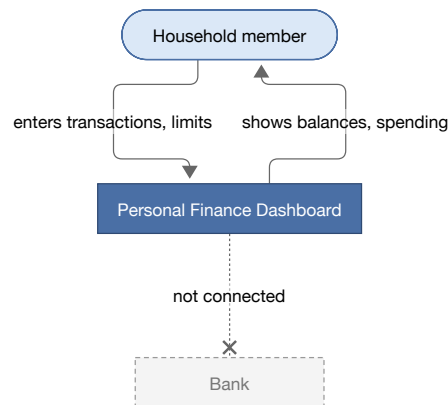


Figure 1: The household member is the only partner. No bank is connected (ch. 2).

Partner	Input	Output
Household member	Household setup, accounts, transactions, monthly limits	Balances, transaction history, category statistics, budget status
Bank	None: Switzerland has no bank-agnostic API (ch. 2)	None

3.2. Technical Context

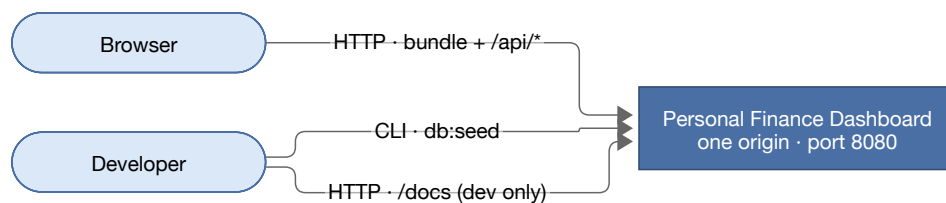


Figure 2: Everything reaches the system through one origin; the developer also uses CLI scripts.

Partner	Channel	Details
Browser	HTTP	One origin (APP_PORT, default 8080): nginx serves the Angular bundle and proxies /api/* (JSON, session cookie) to the API. No TLS in the stack (ch. 11); containers and ports in ch. 7.
Developer	CLI	bun run db:seed, SEED_DEMO at startup, bun run openapi:generate.
Developer	HTTP	Swagger UI /docs, non-production only.

4. Solution Strategy

4.1. Technology and decomposition

Decision	Reason (alternative beaten)
SPA + REST API + SQLite	Three containers (ch. 5) cover a household-sized load; a file database needs no server (beat PostgreSQL).
Modular monolith by feature	backend/src/features/* and frontend/src/app/features/* mirror the domain; one deployable (beat type folders such as controllers/, services/).
Drizzle ORM + drizzle-kit migrations	Typed schema next to its feature; SQL-close queries (beat TypeORM decorators).
better-auth sessions	Cookie sessions and rate limiting out of the box, tables in the same Drizzle schema (beat a hand-written JWT flow). Mounting: ADR-4.
OpenAPI → generated client	@nestjs/swagger spec generates frontend/src/app/core/api/; no duplicated DTOs.
Angular Material 3 + Tailwind 4	Accessible components plus semantic utility tokens (ch. 8 Theming).
Monorepo, Bun	One repository for frontend/ and backend/; Bun as package manager and runtime.

4.2. Quality goals

Quality goal	Approach
Atomic financial writes	SQLite via Drizzle and libsql batch() for multi-statement changes; see ADR-3 in chapter 9 and the money concept in chapter 8.
Responsive navigation	A fixed shell with one scroll panel, boot-time current-route prefetch, then delayed background preloading; see chapter 8.
Low-effort transaction entry	One add button on Home and Transactions; an amount-first dialog with every other field prefilled or optional; see chapter 8 Quick transaction entry and ADR-1.

5. Building Block View

5.1. Level 1: Whitebox Overall System

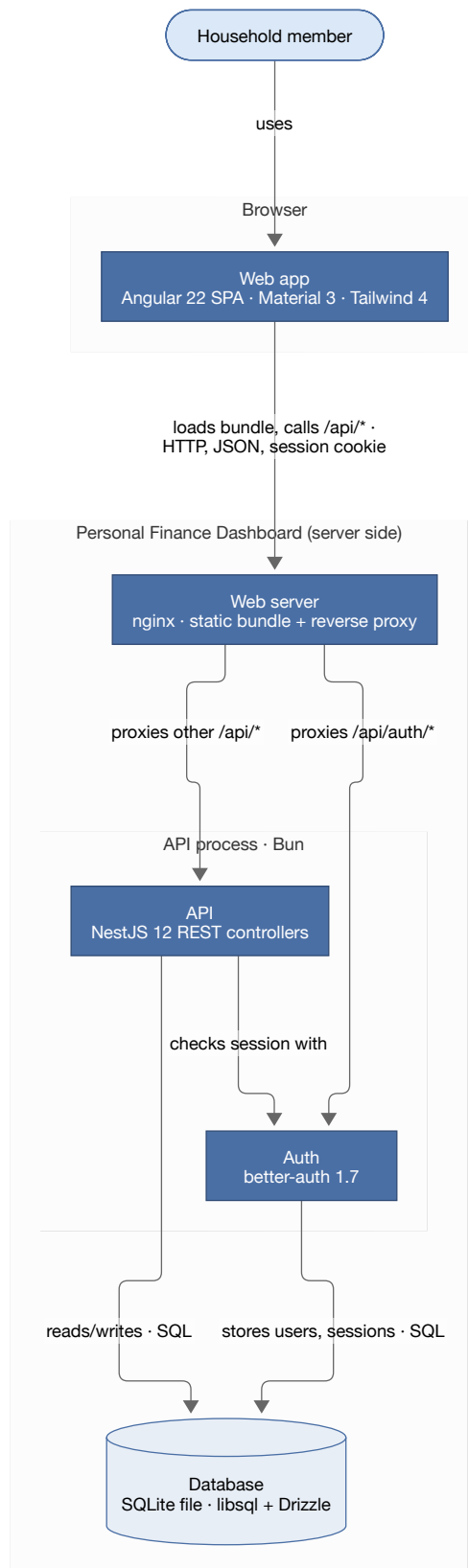


Figure 3: One browser app, one reverse proxy, one API process with auth inside, one SQLite file.

Building block	Responsibility	Source
Web app	Screens, client state, routing, theming	frontend/
Web server	Serves the bundle, proxies /api/, sets X-Real-IP	frontend/nginx.conf.template
API	REST endpoints, business rules, household scoping	backend/src/features/
Auth	Sign-in, session cookies, rate limiting	backend/src/shared/infra/auth/
Database	Persistent state; schema via migrations	backend/drizzle/

5.2. Level 2: Backend Architecture

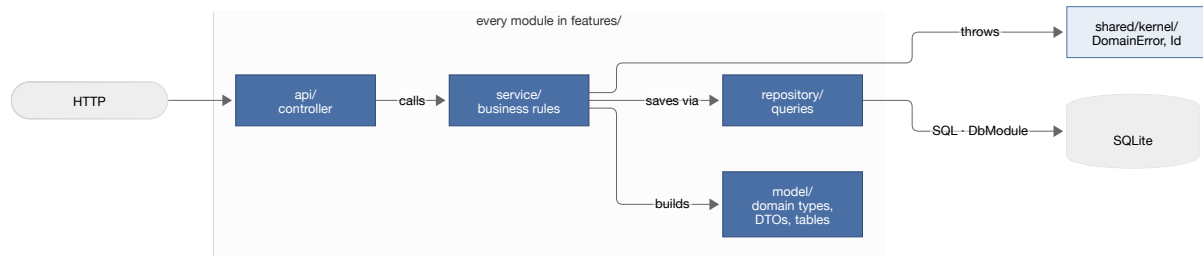


Figure 4: Every feature module has the same four layers: controllers never touch the database, services never see HTTP.

Module	Responsibility
household	Membership and household settings; HouseholdMemberGuard protects every /households/:householdId/* route of the other modules.
onboarding	Wizard validation and atomic household creation (ADR-3).
account	Finance accounts: numbering, archive or delete (M16).
category	Categories, default list, transfer on delete (ADR-1).
transaction	Transactions, filters, statistics.
budget	Monthly limits and take-over (ADR-2); checks categories via category.
shared/infra/	AuthModule (global AuthGuard), DbModule (libsql), HealthModule, setupApp() (CORS, auth handler, validation, error filter; ADR-4).
shared/kernel/	Id, Currency, DomainError; no Nest or Drizzle imports.

New tables must be re-exported from shared/infra/db/schema.ts: drizzle-kit and the better-auth adapter both read the schema from there.

5.3. Level 2: Frontend Architecture

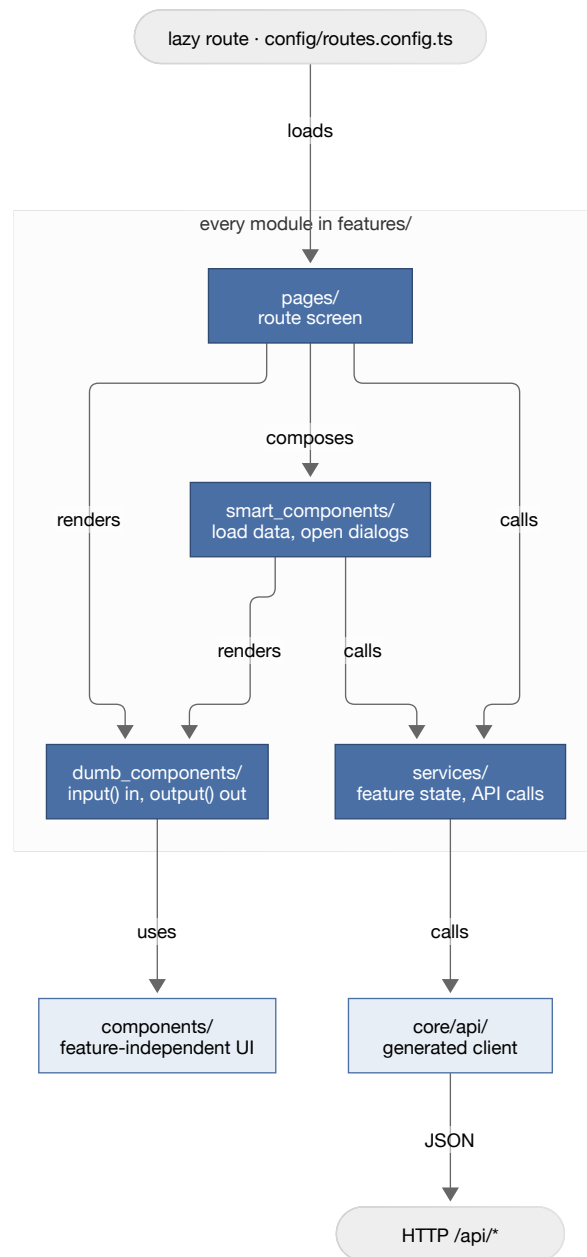


Figure 5: Every feature module has the same folders: dumb components only get inputs and emit outputs; only services call the API (components import just its DTO types).

Module	Responsibility
shell	Authenticated layout, tab navigation, the single scroll panel (ch. 8).
auth	Login page and form.
onboarding	Wizard, localStorage draft, validation calls, final submit.
household	Home page; HouseholdService and the onboarding guards.
transaction	Transactions page, history, add/edit dialog, last-used defaults (ch. 8).

Module	Responsibility
analytics	Analytics tab layout and category statistics chart.
budget	Budgets page, limit dialog, safe-to-spend card, take-over (ADR-2).
settings	Household settings, account and category management, sign-out.
account, category, stats	No own page: components and services reused by Home and Settings.
config/	Routes, prefetch, delayed preload (ch. 8).
core/	Generated API client, better-auth client and AuthGuard, theme, i18n, icons.
components/	Feature-independent UI: dialog service, amount, category avatar, tab nav.

Features may import other features' components and services (Home shows account, budget, and stats components); shared code never imports from a feature (`frontend/AGENTS.md`).

6. Runtime View

6.1. Sign-in and session

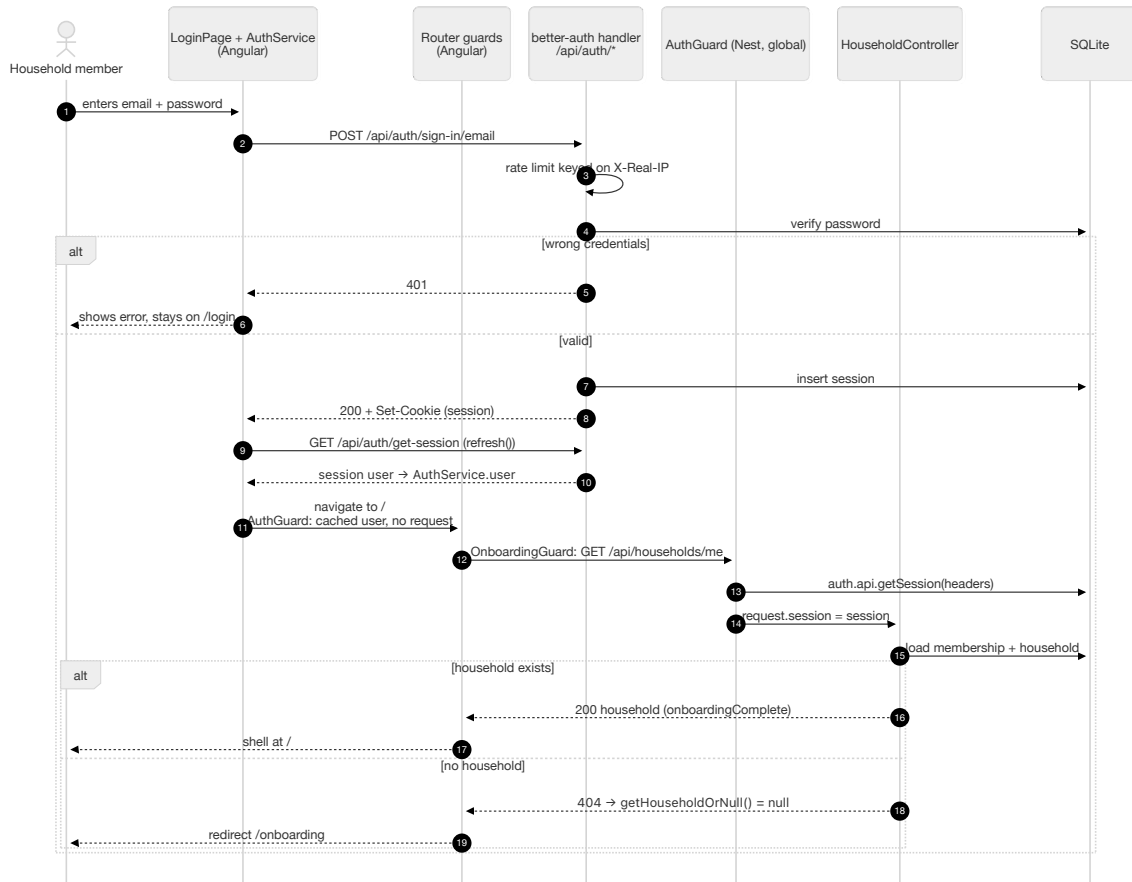


Figure 6: The session lives in an HTTP-only cookie; the SPA never handles a token. Router guards decide between shell and onboarding from GET /api/households/me.

- `signIn()` calls `refresh()` (GET /api/auth/get-session) before navigating, so the Angular AuthGuard passes on the cached user. On a cold boot without session, that same `refresh()` returns no user and the guard redirects to /login.
- /api/households/me answers 404 when the user has no household; `HouseholdService.getHouseholdOrNull()` maps it to null and `OnboardingGuard` redirects to /onboarding.
- Rate limiting keys on X-Real-IP, which nginx overwrites (ch. 7, ch. 8 Security).
- Controller routes need a session unless marked `@Public()`; household routes additionally pass `HouseholdMemberGuard`.

6.2. Onboarding

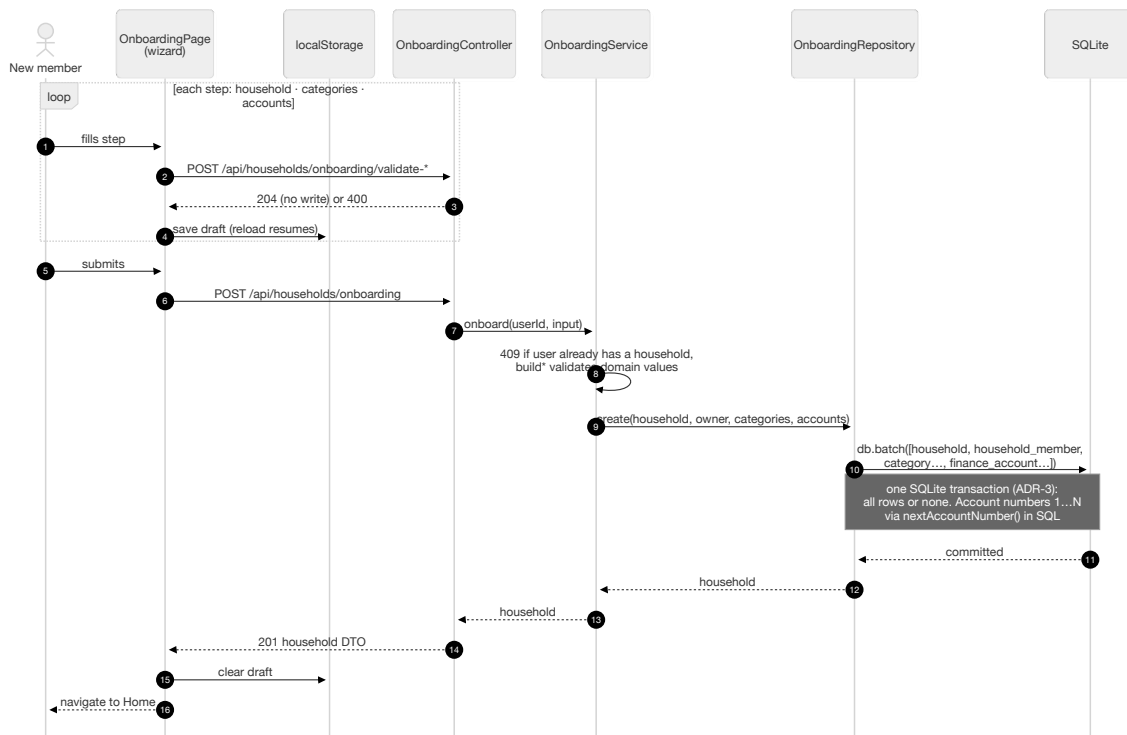


Figure 7: Validation steps never write; the single submit creates the whole household in one libsql batch.

- Steps 2–3 return 204 without a write; a reload resumes from the localStorage draft.
- Only `POST /api/households/onboarding` persists records, atomically (ADR-3). See `onboarding.page.ts` and `OnboardingRepository`.
- A user who already belongs to a household gets 409 (`OnboardingService`).

6.3. Monthly limits take-over



Figure 8: Opening an empty current or next month copies the nearest earlier limits once; a month the user emptied stays empty.

- `BudgetService.loadMonth` (frontend) runs the take-over before the dashboard or budgets tab renders, so both views agree whichever opens first.
- The decision rules and their rationale are in ADR-2.

7. Deployment View

`compose.yaml` starts two containers. Only `web` publishes a host port, as plain HTTP (TLS termination is not part of the stack, ch. 11); `api` is reachable on the Compose network, and its SQLite file is kept on the `db-data` volume.

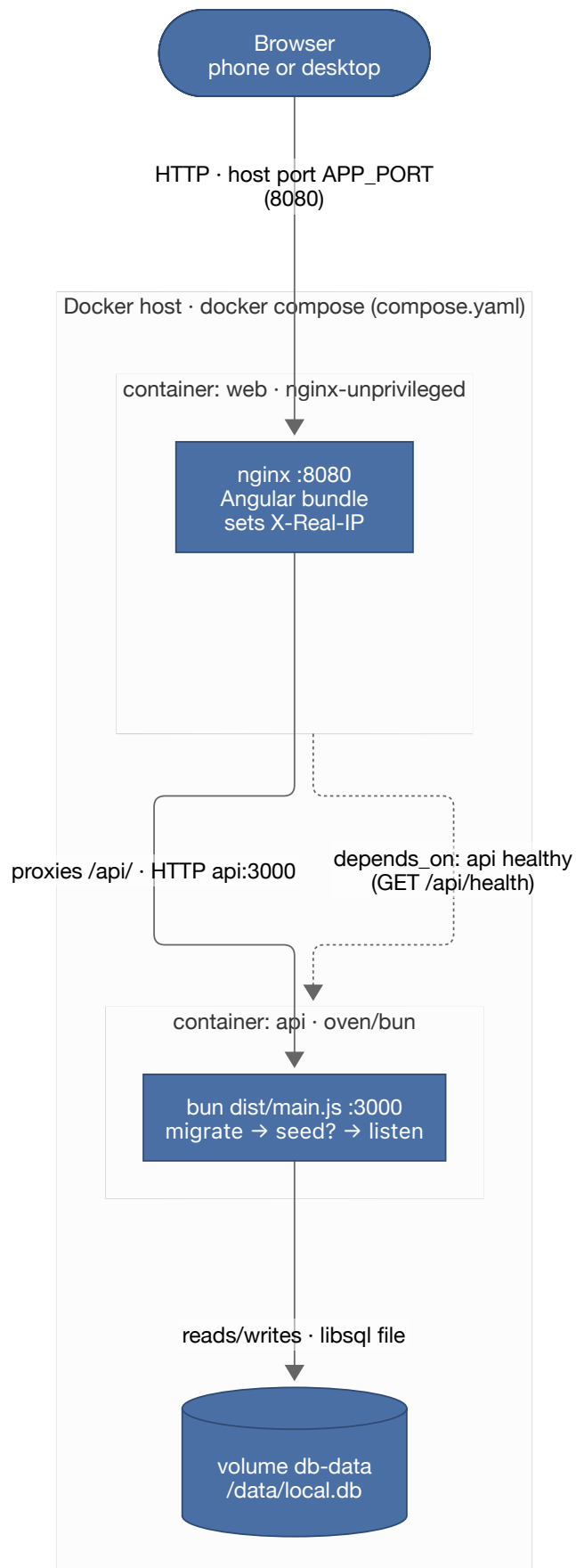


Figure 9: Only nginx is reachable from outside; the API starts after migrations and is healthy before nginx starts.

Node	Artifact and configuration
web	Angular browser bundle in nginx. Host port APP_PORT defaults to 8080; /api/ proxies to API_UPSTREAM=api:3000.
api	Bun/NestJS image, port 3000. DB_FILE_NAME=file:/data/local.db; migrations run at startup before listening.
db-data	Named Docker volume holding the SQLite file across container restarts.

7.1. Runtime configuration

BETTER_AUTH_SECRET is required by Compose and by `env.ts` whenever `NODE_ENV=production`; a fallback secret would make session cookies forgeable. BETTER_AUTH_URL and TRUSTED_ORIGINS must match the browser-facing origin. `main.ts` exposes Swagger /docs and writes OpenAPI only outside production. SEED_DEMO defaults to `true` in Compose, creating the demo and sample users on boot; disable it for a non-demo deployment. In local development it defaults to `false`, and `bun run db:seed` fills the local database explicitly.

nginx overwrites X-Real-IP with `$remote_addr` before proxying. better-auth uses that header for rate limiting; it does not trust the client-controlled X-Forwarded-For chain. If another proxy is placed in front of nginx, `$remote_addr` becomes that proxy's address until a trusted real-IP configuration is added. See the security concept in chapter 8 and `frontend/nginx.conf.template`.

7.2. Development seed

`backend/src/shared/infra/db/seed.ts` creates two sign-in users: `demo` has no household and starts the onboarding wizard; `sample` has a finished household, categories, accounts, budgets, and 200 transactions. The seed is idempotent. The end-to-end backend suite instead migrates and seeds `DB_FILE_NAME=:memory:` (`backend/test/setup-db.ts`).

8. Cross-Cutting Concepts

8.1. Domain model

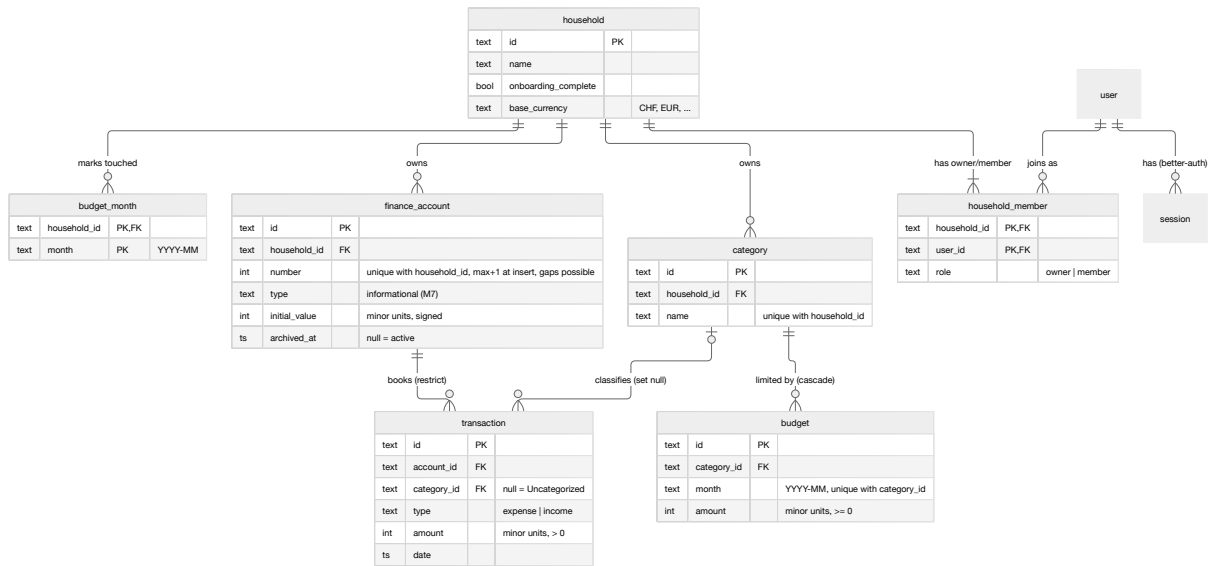


Figure 10: Household is the ownership root; transaction and budget reach it only through their parents. user and session belong to better-auth.

- transaction.category_id null means “Uncategorized” (ADR-1).
- budget_month marks months whose limits were touched (ADR-2).
- The table is finance_account (Drizzle symbol financeAccount) because better-auth already uses account for login providers.

8.2. Household scoping

Financial records belong to a household, directly or through a parent. finance_account and category carry household_id; transaction inherits ownership through its account, and budget through its category. Child tables have no redundant household_id. Reads of child rows follow the parent: TransactionRepository.inHousehold scopes through the account, while budget reads join category and filter category.householdId. Services check parent ownership before creating or changing child rows. The budget_month marker belongs directly to a household. See the schemas and repositories under backend/src/features/.

8.3. Domain errors at the HTTP boundary

Services throw DomainError subclasses from backend/src/shared/kernel/domain-error.ts without importing HTTP exceptions. DomainExceptionFilter in backend/src/shared/infra/errors/ maps their kinds to responses:

Domain error kind	HTTP status
not_found	404
forbidden	403
conflict	409
validation	400

The response contains `statusCode`, the error class name, and its message. This mapping is applied once at the API boundary, leaving domain operations usable outside HTTP.

8.4. Testing

Four suites cover unit, integration, and end-to-end level. All passed on 2026-09-25:

Suite	Tool	Files	Tests	What is tested
Backend unit	Vitest	24	191	Model rules (<code>build*</code> : amounts, months, date ranges, unique names), services, controllers and mappers, guards, env config, <code>DomainExceptionFilter</code> .
Backend integration	Vitest, supertest	10	108	HTTP contract of every feature: auth, onboarding, accounts, categories, transactions and filters, statistics, budgets; household isolation (foreign household's URL or ids).
Frontend unit	Vitest, jsdom	53	277	Services (API calls, onboarding draft, last-used defaults), dumb components (inputs, outputs, forms), smart components and dialogs, guards, theme, <code>i18n</code> , preload strategy.
Browser end-to-end	Playwright	6	8 × 2	Login (anonymous redirect, wrong password), onboarding, account create and rename, transaction add/edit/delete with balance, income and expense on Home, budget overspend; each flow on desktop and mobile.

- **Where:** backend `src/**/*spec.ts` (`bun run test`) and `test/*e2e-spec.ts` (`bun run test:e2e`); frontend `src/**/*spec.ts` (`bun run test`) and `playwright/e2e/` (`bun run e2e`).
- **Fakes over mocks:** unit specs replace collaborators with small fakes typed as `Pick<Repository, 'method'>` or `Pick<Service, 'method'>`, implementing only the methods the subject calls; a cast supplies the full class type.
- **No mocks in integration:** backend integration specs run the real `AppModule`, including `setUpApp()` and `better-auth`, on `:memory:` SQLite migrated and seeded per run (`test/setup-db.ts`).
- **Isolated browser flows:** Playwright starts its own API (`:3100, :memory:`) and `ng serve (:4300)`, in projects `desktop` (Desktop Chrome) and `mobile` (Pixel 7, QS-4). Fixtures in `playwright/fixtures/test.ts` sign up a fresh user, and onboard a household when needed, through the API for each test, so tests share no state and run fully parallel.

8.5. Theming

Angular Material 3 defines `--mat-sys-*` system colors and type tokens in `frontend/src/material-theme.scss`. App-specific income, expense, warning, and account colors use `--app-*` tokens there. `frontend/src/styles.css` maps these tokens to Tailwind names such as `bg-surface-low`, `text-on-surface-variant`, and `text-income`. Templates in `src/app/` use these semantic names, never palette utilities such as `gray-*`, `red-*`, or `green-*`.

Material and app colors use `light-dark()` pairs. `ThemeService` stores `light`, `dark`, or `follow-system` preference per browser and sets `color-scheme` on `<body>`; the browser then resolves the tokens for the selected scheme. Canvas consumers use the service's resolved `light/dark` value because canvas cannot evaluate CSS colors. The Material 3 theme is configured with `mat.theme()` in `material-theme.scss`.

The `category-avatar` returns a CSS `light-dark()` color, while `categorySchemeColor` returns one explicit scheme color for `Chart.js`. `CategoryBarChartComponent` also reads resolved Material ink and border colors from its DOM wrapper. The chart takes `ThemeService.resolved` rather than passing a `light-dark()` expression to canvas (`frontend/src/app/components/category-avatar/category-color.ts`).

8.6. Security

`better-auth` owns sign-in and session cookies; `AuthModule` installs a global guard for Nest controller routes (ADR-4). `setupApp` applies credentialed CORS only to configured `TRUSTED_ORIGINS`. In the Compose deployment, `nginx` is the public entry point and overwrites `X-Real-IP` with its peer address. `better-auth` keys rate limits on that header; using the `forwarded-for` chain would let a client choose its own rate-limit key. Production requires `BETTER_AUTH_SECRET`, and Swagger UI is disabled there (`backend/src/shared/infra/config/env.ts`, `backend/src/main.ts`). Deployment values and the proxy trust boundary are in chapter 7.

8.7. Money and account numbers

Amounts travel through the API and database as integer minor units (cents for the supported currencies); UI inputs convert major units with rounding at submission and display them divided by 100. Account opening balances can be signed, transaction amounts are positive with direction in type, and budget limits are nonnegative. Input DTOs cap a single absolute amount at `MAX_AMOUNT = 100_000_000_000` minor units (one billion major units), leaving headroom under JavaScript's `Number.MAX_SAFE_INTEGER` for balance sums; it is not a cap on the number of transactions. Keep short unit annotations at field boundaries (`backend/src/shared/kernel/currency.ts`).

`finance_account.number` is unique within a household. `nextAccountNumber` computes `coalesce(max(number), 0) + 1` inside the insert statement, so onboarding's ordered `libsql` batch assigns 1 through N without a client-side read/modify/write race. Account creation uses the same expression. See ADR-3 and `backend/src/features/account/model/account.schema.ts`.

8.8. Responsive shell and dialogs

`ShellPage` fills the viewport (`h-dvh`); its `MatTabNavPanel` is the only scroll container. The document has `overscroll-behavior: none`, and the shell uses a bottom navigation bar on phones and tabs under the header from the `md` breakpoint. Keeping page content inside the panel

prevents iOS Firefox from painting it behind its translucent bottom toolbar, which clips fixed elements at the layout viewport (`shell.page.ts`, `material-theme.scss`, `styles.css`; M15).

Form dialogs use the `.app-dialog` CSS class: a full-screen sheet below 640px and a centered card above it, so an already-open dialog responds to resizing. `DialogService.open` synchronously focuses an off-screen input during the tap before the lazy dialog import; iOS then keeps the keyboard open when CDK moves focus to `cdkFocusInitial`. Editing can opt out with `focusInput: false`. The service restores focus to the original trigger after close (`frontend/src/app/components/dialog/dialog.service.ts`).

8.9. Quick transaction entry

A repeat entry needs only the amount (quality goal 3).

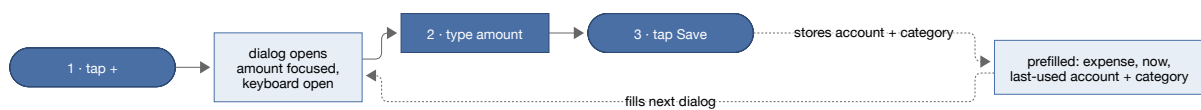


Figure 11: Tap, amount, Save. Everything else is prefilled from the last entry.

- Category and title are optional; no category means “Uncategorized” (ADR-1).
- Code: `TransactionHistoryComponent` (add button), `TransactionFormComponent` (defaults), `TransactionService` (localStorage key `transaction-last-used`).

8.10. Route loading

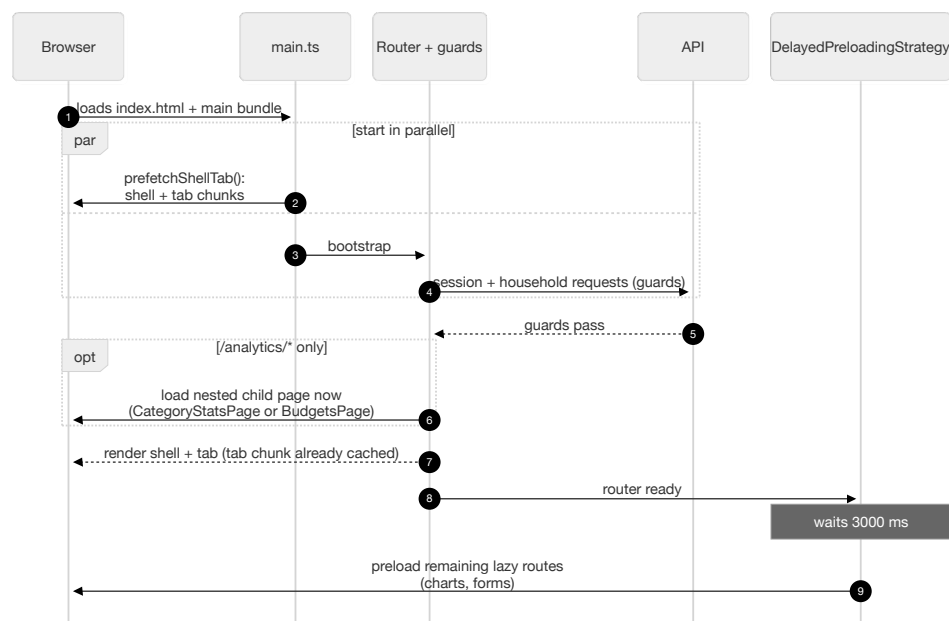


Figure 12: The shell and the tab page download in parallel with the guard requests; everything else waits 3 s.

At bootstrap, `prefetchShellTab(location.pathname)` starts downloading the shell and the page of the first URL segment before auth and household guards finish their requests. For `/analytics/*` that page is the `AnalyticsPage` layout; its nested child (`CategoryStatsPage` or `BudgetsPage`) loads only after the guards pass. After router startup, `DelayedPreloadingStrategy` waits 3000 ms before loading each other lazy route, so chart and form chunks do not compete with the first page on a slow connection. Cached, content-hashed bundles then make later tab switches cheaper (`frontend/src/main.ts`, `config/routes.config.ts`, `config/delayed-preloading.strategy.ts`; quality scenario QS-2 in chapter 10).

9. Architecture Decisions (ADRs)

9.1. ADR-1: “Uncategorized” is the absence of a category, not a reserved category row

Status: accepted · deviates from the wording of M5 and M6 in the proposal.

9.1.1. Context

M5 lets a transaction be entered without choosing a category, M6 lets a category be deleted without leaving its transactions unassigned. The proposal describes both with a *reserved catch-all category* that such transactions fall into.

A reserved category would be an ordinary row in `category` with special rules: created with every household, not renamable, not deletable, not selectable as a transfer target of itself, excluded from the suggested onboarding list, and recognisable by a flag or a magic name in every query that treats it differently.

9.1.2. Decision

`transaction.category_id` is nullable. `null` is the catch-all: a transaction without a category belongs to the bucket “Uncategorized”. The foreign key uses `on delete set null`, so deleting a category moves its transactions into that bucket in the same statement, unless the user picks another category to transfer them to.

The bucket is a first-class value everywhere a category appears: the transaction list and the analytics chart label it “Uncategorized”, the history filter offers it as an option (`categoryId=none`), and the category statistics group it as its own slice.

9.1.3. Consequences

- M5 and M6 hold as the user experiences them: entry is never blocked, and deleting a category offers “move to Uncategorized” or “move to another category”. No transaction is ever without a visible assignment.
- No special-case rows: nothing to seed, protect, migrate or filter out of the category management UI. The database enforces the fallback by itself.
- The bucket cannot be renamed; its label comes from the translation files.
- Limits are keyed by `(category_id, month)`, so uncategorized spending cannot get a limit of its own. It still counts as an expense in the monthly cashflow and therefore lowers Safe-to-Spend. No Should-have story asks for a limit on it.
- Should a limit on uncategorized spending become a requirement, the migration is mechanical: insert one reserved category per household and update the `null` rows to it.

9.2. ADR-2: A new month takes over the nearest earlier limits, automatically only for the current and next month

Status: accepted · deviates from the wording of S4 in the proposal.

9.2.1. Context

S4 asks that a new month takes over the limits of the *previous month*, can be changed there, and leaves the previous month unchanged, so that limits need not be entered again every month.

Three questions are left open by that sentence:

- What counts as the previous month when that month has no limits, for example because the app was not used in August?

- When is a month “new”? The system has no scheduler, and a month nobody opens needs no limits.
- What happens to a month the user emptied on purpose? S1 lets every limit be removed, S3 lets a month stay deliberately unbudgeted. A rule that refills every empty month undoes both.

9.2.2. Decision

- **Copies, not a fallback.** Taking over inserts independent budget rows for the target month. Reading a month never looks at other months, so editing a copy cannot change its source, and “no row” keeps meaning “no limit” (S3).
- **Source is the nearest earlier month that has limits**, not strictly the calendar month before. One unused month does not lose the plan.
- **Automatic only for the current and the next calendar month**, on first view (POST .../:month/copy-previous?auto=true). Every other empty month offers an explicit “Take over previous limits” action. The target month must be empty (409 otherwise).
- **Touched months never fill themselves again.** Table `budget_month(household_id, month)` records every month whose limits were set, removed or taken over. The automatic take-over skips such a month, so removing the last limit leaves it empty. The explicit action still fills it.

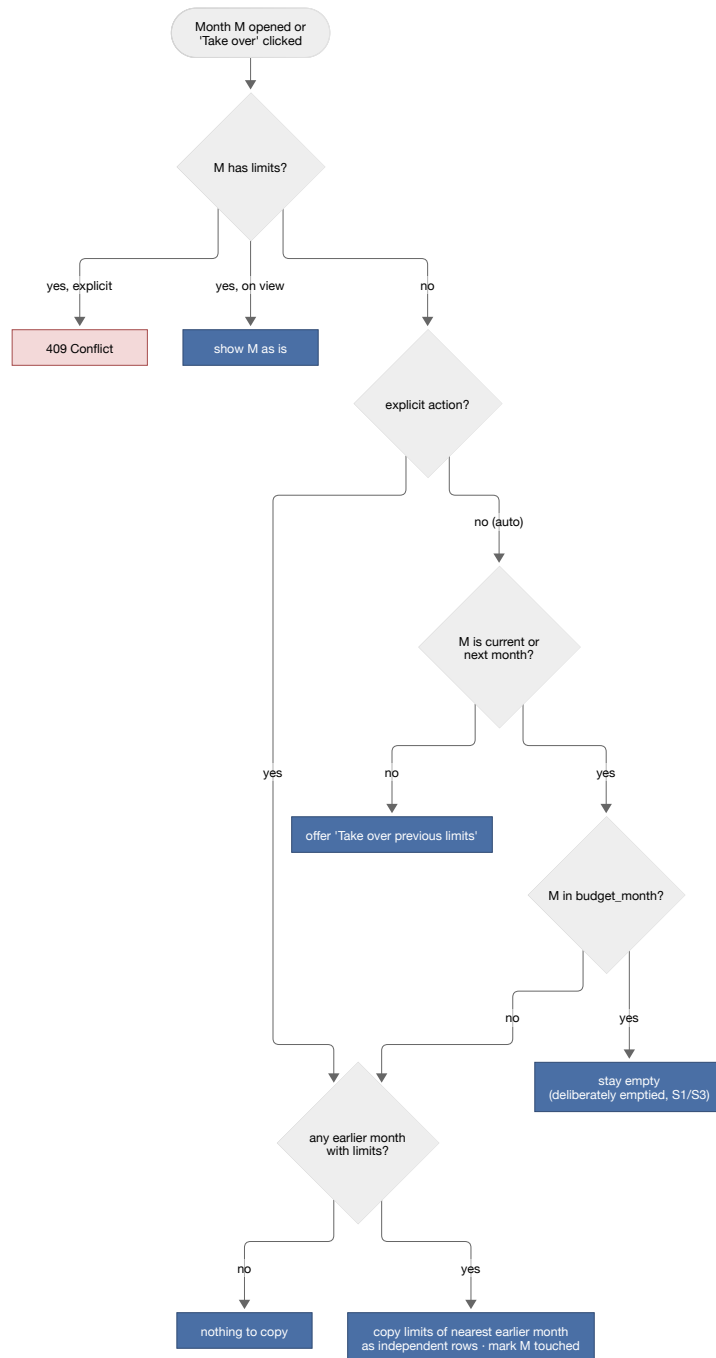


Figure 13: Take-over decision as implemented in *BudgetService.copyFromPrevious* (backend) and *loadMonth* (frontend).

9.2.3. Consequences

- S4 holds for the common case: opening a new month shows last month's limits, and changing them leaves that month as it was.
- S1 and S3 hold together with S4: a month emptied on purpose stays empty until the user asks otherwise.
- A month nobody opened is not filled and costs nothing; past months are filled only on request.
- Skipping is decided per month only. If August was emptied on purpose, September still takes over the limits of July, the nearest month that has any. Treating an emptied month as

“inherit nothing” would help this edge case and lose the plan after every unused month, so it was rejected.

- The trigger is the client clock (current/next month), since the server knows no household timezone. A client with a wrong clock can at most fill a month early; the user can remove the copies.
- `budget_month` rows are never deleted on their own; they go with the household.

9.3. ADR-3: Use libsql batch() for atomic multi-statement writes

Status: accepted.

9.3.1. Context

Onboarding inserts a household, owner membership, categories, and accounts together. Category transfer updates transactions before deleting the category; a household currency change updates accounts with the household. Sequential statements could leave partial state. In the libsql in-memory e2e database, `db.transaction` swaps the client's connection after an explicit transaction and loses that database.

9.3.2. Options

Option	Trade-off
<code>db.transaction</code>	Natural Drizzle API, but the libsql connection swap breaks <code>:memory:</code> e2e state.
Sequential writes	Simple, but a later failure can leave a partial household or transfer.
<code>db.batch</code>	libsql executes the statements in one transaction and rolls back together; statements must be assembled before execution.

9.3.3. Decision

Use `db.batch()` in `OnboardingRepository`, `CategoryRepository.deleteCategory` when transferring, and `HouseholdRepository.update` when changing currency. The seed also batches related rows. `AccountRepository.createAccount` is a single insert, not a batch; its `nextAccountNumber` SQL expression also works for ordered account inserts inside the onboarding batch.

9.3.4. Consequences

- These writes remain atomic in production and against the in-memory e2e database.
- Operations that need a result to choose the next statement cannot use this prepared batch shape; they need a separate design.
- Revisit if libsql fixes explicit-transaction connection handling or the persistence driver changes.

9.4. ADR-4: Mount better-auth's HTTP handler before Nest initializes

Status: accepted.

9.4.1. Context

better-auth's Node handler needs the raw request body for `/api/auth/*`. Nest's default body parser consumes that stream during initialization. Auth endpoints also need CORS headers even when the handler responds without entering Nest's controller pipeline.

9.4.2. Options

Option	Trade-off
Nest controller wrapper	Would run after body parsing and hand the handler a consumed stream.
Disable Nest body parsing	Preserves the auth stream but requires manual parser wiring for ordinary controllers.
Early Express middleware	Keeps Nest defaults for controllers while giving better-auth the raw stream.

9.4.3. Decision

`setupApp` registers CORS, then `mountAuthHandler`, before `app.init()` or `listen()`. The middleware handles `/api/auth/*` directly and calls `next()` for other paths. `AuthModule` remains inside Nest to expose the auth instance and install the global controller guard.

9.4.4. Consequences

- Requests take two paths: `better-auth` answers `/api/auth/*` and owns validation and responses there; all other requests pass Nest's body parser, guards, validation pipe, and error filter.
- The order in `app.setup.ts` and `auth.handler.ts` is part of the contract and must stay beside the code.
- Revisit if Nest or `better-auth` gains an integration that preserves the raw stream and response behavior.

10. Quality Requirements

10.1. Quality Tree

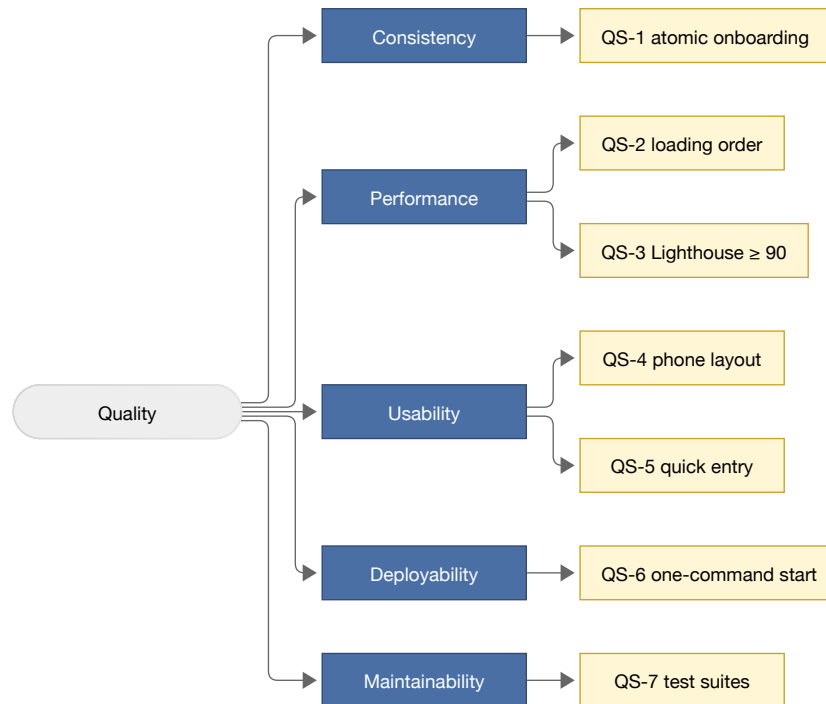


Figure 14: Five aspects, seven scenarios. QS-1, QS-2, QS-4, and QS-5 check the quality goals of chapter 1; the others check the lecturer's requirements.

10.2. Quality Scenarios

Quality	Steps to reproduce	Expected outcome
Consistency		
QS-1 Atomic onboarding (goal 1)	1. Sign in as <code>demo@finance.local</code> in local development. 2. In <code>backend/local.db</code>, add a trigger that aborts inserts into <code>finance_account</code> (<code>raise(abort, ...)</code>). 3. Finish the onboarding wizard, then drop the trigger.	Submit fails with 500. No household, household_member, or category row exists for the user; the wizard draft is still in localStorage.
Performance		
QS-2 Loading order (goal 2)	1. Sign in at <code>http://localhost:8080</code>. 2. DevTools → Network: disable cache, throttle to "Slow 4G". 3. Reload <code>/transactions</code>.	Shell and transactions chunks start before <code>get-session</code> and <code>households/me</code> return. Other route chunks start at least 3 s after the page renders.

Quality	Steps to reproduce	Expected outcome
QS-3 Lighthouse	1. docker compose up --build, sign in as sample@finance.local. 2. DevTools → Lighthouse, all four categories, on Mobile and Desktop for each shell tab.	Average of Performance, Accessibility, Best Practices, and SEO is at least 90 for Mobile and for Desktop.
Usability		
QS-4 Phone layout (goal 2)	1. Open /transactions on a phone (iOS Firefox) or in the Playwright mobile project. 2. Scroll to the end of the list.	Bottom navigation stays visible; the document does not scroll, only the tab panel does.
QS-5 Quick entry (goal 3)	1. On a phone, add one expense with account and category. 2. Tap +, type an amount, tap Save.	Keyboard is open when the dialog appears; account and category match the previous entry; the entry saves without touching another field.
Deployability		
QS-6 One-command start	1. Fresh clone: cp .env.example .env, set BETTER_AUTH_SECRET. 2. docker compose up --build. 3. Sign in at http://localhost:8080 as demo@finance.local / demo-password, enter data. 4. docker compose restart.	Both containers report healthy; the demo user reaches onboarding; data entered before the restart is still there.
Maintainability		
QS-7 Test suites	1. Backend: bun run test, bun run test:e2e. 2. Frontend: bun run test, bun run e2e.	All four suites pass (scope and counts in ch. 8 Testing).

10.3. Lighthouse Results

Production bundle via docker compose (port 8080), 2026-09-25. Every run meets QS-3.

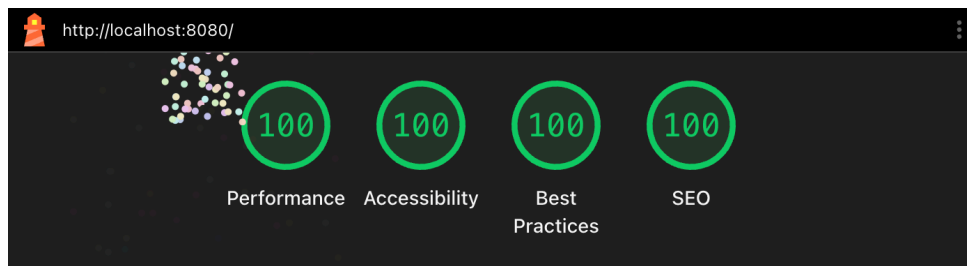


Figure 15: Home /, Desktop: average 100.

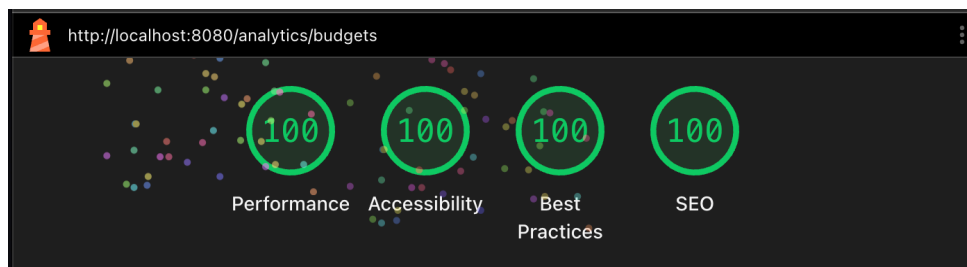


Figure 16: Budgets /analytics/budgets, Desktop: average 100.

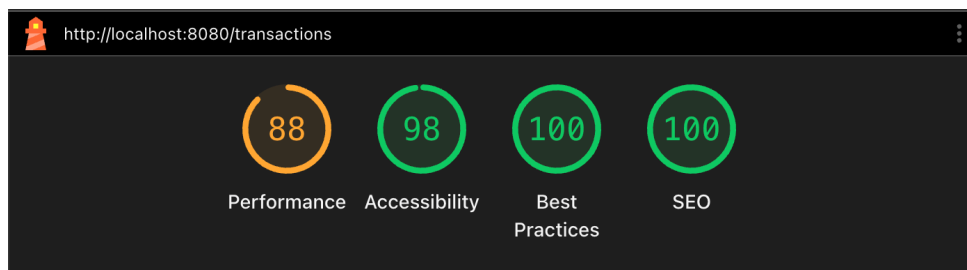


Figure 17: Transactions /transactions, Mobile: average 96.5 (Performance 88, Accessibility 98).

11. Risks and Technical Debt

Risk / debt	Impact	Mitigation
No TLS in the Compose stack	nginx serves plain HTTP on 8080; outside localhost, credentials and the session cookie travel unencrypted.	Terminate TLS in front of nginx (with trusted real-IP, see proxy row) and set BETTER_AUTH_URL/ TRUSTED_ORIGINS to the https origin.
Atomicity not proven by fault injection (QS-1)	A regression to sequential writes would pass existing tests.	Add a failing-statement test for the onboarding batch.
SQLite single writer, one API instance	No horizontal scaling; concurrent writes serialize.	Adequate for household load; switching driver revisits ADR-3.
Proxy trust boundary	An extra proxy in front of nginx makes all clients share one rate-limit key.	Configure trusted real-IP in nginx before adding a proxy (ch. 7).
Client clock drives auto take-over	A wrong clock may fill a month early.	User removes copies; see ADR-2.
better-auth CLI schema generation	Generated auth.schema.ts can lag the library version.	Review generated diff after bun run auth:generate.

12. Glossary

Term	Definition
Household	Ownership root of all financial data; has one base currency and owner/member users.
Finance account	A money container (checking, savings, cash, ...) with opening balance; table <code>finance_account</code> .
Transaction	Positive amount of type <code>expense</code> or <code>income</code> booked on one finance account.
Uncategorized	Bucket of transactions without category (<code>category_id</code> null), ADR-1.
Limit (budget)	Spending cap of one category for one month <code>YYYY-MM</code> ; no row = no limit.
Take-over	Copying the nearest earlier month's limits into an empty month, ADR-2.
Touched month	Month recorded in <code>budget_month</code> ; never auto-filled again.
Minor units	Integer cents used for every stored amount.
Shell	Authenticated layout with tab navigation and the single scroll panel.